



MODUL PRAKTIKUM

Disusun Oleh:

Aprilia Kurnianti, S.T., M.Eng.

Dosen Pengampuh Matkul IBD

PRAKTIKUM 8

Transaction Management, Locking, Deadlock, and Isolation Level

Tujuan Praktikum

Setelah menyelesaikan praktikum ini mahasiswa mampu:

1. Menjelaskan konsep transaksi pada database.
2. Menerapkan prinsip ACID.
3. Gunakan BEGIN TRANSACTION, COMMIT, dan ROLLBACK.
4. Gunakan SAVE TRANSACTION untuk membuat savepoint.
5. Mengamati mekanisme locking pada SQL Server.
6. Menjelaskan hubungan transaksi dengan locking.
7. Menggunakan berbagai Isolation Level.
8. Mensimulasikan kondisi deadlock.
9. Mengidentifikasi penyebab deadlock.
10. Menerapkan strategi sederhana untuk mengurangi kemungkinan terjadinya deadlock.

Persiapan

Gunakan database DB_Petshop.

Praktikum 1 - Periksa tabel:

```
SELECT * FROM dbo.Pelanggan;  
  
SELECT * FROM dbo.Penjualan;  
  
SELECT * FROM dbo.DetailPenjualan;  
  
SELECT * FROM dbo.Produk;
```

Jika nama tabel pada database mahasiswa berbeda, sesuaikan nama tabel dan kolomnya.

Praktikum 2 - Mengenal Transaction dan Commit

```
BEGIN TRANSACTION;  
  
UPDATE dbo.Produk  
SET Stok = Stok - 1  
WHERE KodeProduk = 'PRD001';  
  
COMMIT TRANSACTION;
```

Perhatikan bahwa perubahan belum dianggap selesai sampai transaksi di-COMMIT.

Tugas

1. Jalankan transaksi tersebut.
2. Periksa perubahan data.
3. Jelaskan fungsi BEGIN TRANSACTION.
4. Jelaskan fungsi COMMIT.
5. Buat transaksi yang mengubah salah satu tabel database, lalu simpan dengan COMMIT.

Praktikum 3 – Rollback

```
BEGIN TRANSACTION;

UPDATE dbo.Produk
SET Stok = Stok - 5
WHERE KodeProduk = 'PRD001';

SELECT *
FROM dbo.Produk
WHERE KodeProduk = 'PRD001';

ROLLBACK TRANSACTION;

SELECT *
FROM dbo.Produk
WHERE KodeProduk = 'PRD001';
```

Perhatikan perbedaan data sebelum dan sesudah ROLLBACK.

Tugas

Buktikan bahwa perubahan yang dilakukan dalam transaksi dapat dibatalkan.

Praktikum 4 - Membandingkan COMMIT dan ROLLBACK

Lakukan dua percobaan.

Percobaan A

```
BEGIN TRANSACTION;

UPDATE dbo.Produk
SET Stok = Stok - 2
WHERE KodeProduk = 'PRD001';

COMMIT TRANSACTION;
```

Percobaan B

```
BEGIN TRANSACTION;

UPDATE dbo.Produk
SET Stok = Stok - 2
WHERE KodeProduk = 'PRD001';

ROLLBACK TRANSACTION;
```

Tugas

Buat tabel pengamatan:

Percobaan	Perintah Akhir	Perubahan Disimpan?
A	COMMIT	...
B	ROLLBACK	...

Praktikum 5 - Transaction pada Beberapa Operasi

```

BEGIN TRANSACTION;

UPDATE dbo.Produk
SET Stok = Stok - 2
WHERE KodeProduk = 'PRD001';

INSERT INTO dbo.Penjualan
(
    KodePenjualan,
    TanggalPenjualan,
    KodePelanggan,
    KodePegawai,
    MetodeBayar
)
VALUES
(
    'PJI999',
    GETDATE(),
    'PLG001',
    'PGW101',
    'Cash'
);

COMMIT TRANSACTION;

```

Tugas

Buat transaksi yang minimal melakukan 2 operasi pada 2 tabel yang saling berhubungan.

Kemudian jelaskan:

Mengapa kedua operasi tersebut sebaiknya dilakukan dalam satu transaksi?

Praktikum 6 - SAVE TRANSACTION

```

BEGIN TRANSACTION;

UPDATE dbo.Produk
SET Stok = Stok - 1
WHERE KodeProduk = 'PRD001';

SAVE TRANSACTION Titik1;

UPDATE dbo.Produk
SET Stok = Stok - 2
WHERE KodeProduk = 'PRD002';

ROLLBACK TRANSACTION Titik1;

COMMIT TRANSACTION;

```

Perhatikan bahwa:

- perubahan sebelum Titik1 tetap;
- perubahan setelah Titik1 dibatalkan;
- Transaksi kemudian dapat di-COMMIT.

Tugas

Buat transaksi yang memiliki:

1. operasi pertama;
2. SAVE TRANSACTION;
3. operasi kedua;
4. ROLLBACK ke savepoint;
5. COMMIT.

Kemudian buktikan hasilnya.

Praktikum 7 - SAVE TRANSACTION dengan Beberapa Savepoint

```
BEGIN TRANSACTION;

-- Tahap 1
UPDATE dbo.Produk
SET Stok = Stok - 1
WHERE KodeProduk = 'PRD001';

SAVE TRANSACTION Tahap1;

-- Tahap 2
UPDATE dbo.Produk
SET Stok = Stok - 2
WHERE KodeProduk = 'PRD002';

SAVE TRANSACTION Tahap2;

-- Tahap 3
UPDATE dbo.Produk
SET Stok = Stok - 3
WHERE KodeProduk = 'PRD003';

ROLLBACK TRANSACTION Tahap2;

COMMIT TRANSACTION;
```

Tugas

Amati hasilnya dan tentukan perubahan mana yang:

- tetap disimpan;
- dibatalkan.

Praktikum 8 - Melihat Informasi

Locking digunakan oleh SQL Server untuk mengatur akses beberapa transaksi terhadap data yang sama.

Gunakan dua session.

SESSION 1

```
BEGIN TRANSACTION;

UPDATE dbo.Produk
SET Stok = Stok - 1
WHERE KodeProduk = 'PRD001';
```

Jangan jalankan COMMIT atau ROLLBACK terlebih dahulu.

SESSION 2

Pada Query Window kedua:

```
SELECT *  
FROM dbo.Produk  
WHERE KodeProduk = 'PRD001';
```

Perhatikan apakah query pada Session 2 langsung mendapatkan hasil atau harus menunggu, tergantung pada isolation level dan jenis operasi yang dilakukan.

Setelah selesai:

SESSION 1

```
ROLLBACK TRANSACTION;
```

Praktikum 9 – Melihat Informasi Lock

```
SELECT  
    session_id,  
    status,  
    command,  
    wait_type,  
    wait_time,  
    blocking_session_id  
FROM sys.dm_exec_requests  
WHERE session_id <> @@SPID;  
  
SELECT  
    request_session_id,  
    resource_type,  
    resource_database_id,  
    request_mode,  
    request_status  
FROM sys.dm_tran_locks;
```

Tugas

Identifikasi:

- session yang sedang aktif;
- session yang menunggu;
- jenis resource;
- mode lock;
- status lock.

Praktikum 10 - READ UNCOMMITTED

```
SET TRANSACTION ISOLATION LEVEL READ UNCOMMITTED;  
  
BEGIN TRANSACTION;  
  
SELECT *  
FROM dbo.Produk  
WHERE KodeProduk = 'PRD001';  
  
COMMIT TRANSACTION;
```

Isolation level ini memungkinkan transaksi membaca data yang belum dikommit oleh transaksi lain. Fenomena ini dikenal sebagai **dirty read**.

Praktikum 11 - READ COMMITTED

```
SET TRANSACTION ISOLATION LEVEL READ COMMITTED;

BEGIN TRANSACTION;

SELECT *
FROM dbo.Produk
WHERE KodeProduk = 'PRD001';

COMMIT TRANSACTION;
```

isolation level yang umum digunakan sebagai default SQL Server.

Praktikum 12 - REPEATABLE READ

```
SET TRANSACTION ISOLATION LEVEL REPEATABLE READ;

BEGIN TRANSACTION;

SELECT *
FROM dbo.Produk
WHERE KodeProduk = 'PRD001';

-- tunggu dan lakukan pengujian dari Session 2

COMMIT TRANSACTION;
```

Praktikum 13 - SERIALIZABLE

```
SET TRANSACTION ISOLATION LEVEL SERIALIZABLE;

BEGIN TRANSACTION;

SELECT *
FROM dbo.Produk
WHERE Stok > 0;

-- lakukan pengujian dari Session 2

COMMIT TRANSACTION;
```

PRAKTIKUM 14 - Simulasi Deadlock

Deadlock harus ditangani dengan hati-hati dan hanya pada database praktikum.

Gunakan dua session.

SESSION 1

```

BEGIN TRANSACTION;

UPDATE dbo.Produk
SET Stok = Stok - 1
WHERE KodeProduk = 'PRD001';

WAITFOR DELAY '00:00:05';

UPDATE dbo.Produk
SET Stok = Stok - 1
WHERE KodeProduk = 'PRD002';

COMMIT TRANSACTION;

```

SESSION 2

Jalankan hampir bersamaan:

```

BEGIN TRANSACTION;

UPDATE dbo.Produk
SET Stok = Stok - 1
WHERE KodeProduk = 'PRD002';

WAITFOR DELAY '00:00:05';

UPDATE dbo.Produk
SET Stok = Stok - 1
WHERE KodeProduk = 'PRD001';

COMMIT TRANSACTION;

```

PRAKTIKUM 15 - Integrasi Transaction + TRY...CATCH

Gunakan materi sebelumnya untuk membuat transaksi dengan error handling.

```

BEGIN TRY

    BEGIN TRANSACTION;

    UPDATE dbo.Produk
    SET Stok = Stok - 1
    WHERE KodeProduk = 'PRD001';

    -- proses lainnya

    COMMIT TRANSACTION;

END TRY

BEGIN CATCH

    IF @@TRANCOUNT > 0
        ROLLBACK TRANSACTION;

    SELECT
        ERROR_NUMBER() AS ErrorNumber,
        ERROR_MESSAGE() AS ErrorMessage;

END CATCH;

```

Tugas

Buat satu proses transaksi pada database Anda yang menggabungkan:

- Transaction;

- COMMIT;
- ROLLBACK;
- TRY...CATCH;
- validasi bisnis.